



UNIVERZITET U BEOGRADU
MAŠINSKI FAKULTET – MATEMATIČKI FAKULTET
Industrija 4.0

Računarska inteligencija

Poređenje arhitektura dubokih neuronskih mreža za klasifikaciju satelitskih snimaka

ResNet, DenseNet, DPN i Vision Transformer na EuroSAT skupu podataka

Projektni rad

Student:

Dorđe Marković, 4003/2023
4003-2023@studenti.mas.bg.ac.rs

Predmetni nastavnik:

prof. dr Aleksandar Kartelj

Asistent:

asist. Stefan Kapunac

Beograd, 2026.

Sažetak

U radu sam uporedio četiri arhitekture dubokih neuronskih mreža na zadatku klasifikacije satelitskih snimaka iz skupa EuroSAT: ResNet18, prilagođenu arhitekturu zasnovanu na DenseNet121 rasporedu blokova, pojednostavljeni Dual Path Network (DPN) i manju varijantu Vision Transformera (ViT). Sve modele sam implementirao u Pythonu pomoću biblioteke PyTorch i trenirao pod istim osnovnim uslovima: na istoj podeli podataka, sa istom normalizacijom, brojem epoha i optimizatorom. U tabelama i grafikonima koristim kraće oznake DenseNet121 i DPN.

EuroSAT sadrži 27.000 RGB slika dimenzije 64×64 piksela, raspoređenih u 10 klasa zemljišnog pokrivača. Za trening je korišćeno 60% slika, a za validaciju i test po 20%. Modeli su trenirani 10 epoha optimizatorom Adam, sa stopom učenja 0,001 i veličinom grupe 64. Najbolji rezultat na test skupu ostvario je prilagođeni DenseNet121 sa tačnošću od 91,81%, neposredno ispred DPN modela sa 91,67%. ResNet18 je ostvario 87,56%, dok je kompaktni ViT ostvario 86,37%.

Najbolje test tačnosti dobili su DenseNet i DPN. ViT je imao daleko najmanje parametara, ali je bez prethodnog treniranja i posle samo deset epoha ostao iza konvolucionih modela. Pored ukupne tačnosti, uporedio sam validacione krive, matrice konfuzije i mere precision, recall i F1 za DPN i ViT.

Ključne reči: duboko učenje, EuroSAT, satelitski snimci, ResNet, DenseNet, DPN, Vision Transformer, klasifikacija slika.

Sadržaj

Sažetak	1
1 Uvod	5
1.1 Predmet i cilj rada	5
1.2 Organizacija rada	5
2 Teorijska osnova	6
2.1 Konvolucione neuronske mreže	6
2.2 Rezidualno učenje i ResNet	6
2.3 Guste veze i DenseNet	7
2.4 Dual Path Network	7
2.5 Vision Transformer	7
2.6 Funkcija greške i mere kvaliteta	8
3 Skup podataka i pretprocesiranje	9
3.1 EuroSAT skup podataka	9
3.2 Podela na trening, validaciju i test	9
3.3 Pretvaranje slika i normalizacija	10
3.4 Augmentacija trening podataka	11
3.5 DataLoader i mini-grupe	11
4 Implementacija modela	12
4.1 ResNet18	12
4.2 Prilagođeni DenseNet121	12
4.3 DPN	13
4.4 Kompaktni Vision Transformer	13
4.5 Poređenje veličine modela	14
5 Postavka eksperimenta i treniranje	14
5.1 Softversko i hardversko okruženje	14
5.2 Zajednički hiperparametri	15
5.3 Jedna epoha treninga	15
5.4 Validacija i čuvanje najboljeg modela	16
5.5 Ponovljivost eksperimenta	17
6 Rezultati i analiza	17
6.1 Kretanje validacione tačnosti	17
6.2 Konačna test tačnost	18
6.3 Matrice konfuzije	19

6.4	Detaljniji rezultati DPN i ViT modela	19
6.5	Primeri DPN predikcija	21
6.6	Odnos tačnosti i broja parametara	22
6.7	Diskusija glavnih nalaza	22
6.8	Ograničenja	23
7	Zaključak	23
A	Struktura projekta	25
B	Tačnosti po epohama	26

Spisak slika

1	Pojednostavljen prikaz osnovnog rezidualnog bloka.	6
2	Osnovna ideja dve putanje kod DPN modela.	7
3	Tok podataka kroz korišćeni Vision Transformer.	8
4	Validaciona tačnost sva četiri modela po epohama.	17
5	Poređenje test tačnosti modela.	18
6	Normalizovane matrice konfuzije za sva četiri modela.	20
7	Primeri tačnih DPN predikcija na test slikama.	22

Spisak tabela

1	Klase u EuroSAT RGB skupu.	9
2	Podela EuroSAT skupa korišćena u eksperimentu.	9
3	Broj parametara implementiranih modela.	14
4	Hiperparametri korišćeni za sva četiri modela.	15
5	Glavni rezultati eksperimenta.	18
6	Rezultati DPN modela po klasama na test skupu.	19
7	Rezultati ViT modela po klasama na test skupu.	21
8	Grubo poređenje tačnosti i veličine modela.	22
9	Validaciona tačnost po epohama u procentima.	26

1 Uvod

Satelitski snimci se koriste u poljoprivredi, urbanističkom planiranju, praćenju šuma, proceni štete nakon prirodnih nepogoda i mnogim drugim oblastima. Jedan od osnovnih zadataka je automatsko određivanje klase zemljišta na osnovu slike. Ručna obrada velikog broja snimaka zahteva dosta vremena, pa su metode mašinskog i dubokog učenja veoma korisne za ovaj problem.

Konvolucione neuronske mreže su dugo predstavljale glavni pristup za klasifikaciju slika. Kasnije su uvedene rezidualne veze, guste veze između slojeva i druge ideje čiji je cilj da omoguće treniranje dubljih mreža. U novije vreme su i transformer arhitekture, prvobitno razvijene za obradu teksta, prilagođene radu sa slikama. Zbog toga je korisno uporediti različite ideje na istom skupu i pod istim uslovima, umesto da se posmatraju samo rezultati iz različitih radova.

1.1 Predmet i cilj rada

Predmet rada je višeklasna klasifikacija RGB satelitskih snimaka iz EuroSAT skupa. Svaka slika pripada jednoj od deset klasa, na primer šuma, reka, industrijska zona ili stambena zona. Model kao izlaz daje deset brojeva, po jedan za svaku klasu, a kao konačna predikcija uzima se klasa sa najvećim izlazom.

Glavni cilj je praktično poređenje sledeće četiri arhitekture:

- ResNet18, kao predstavnik rezidualnih konvolucionih mreža;
- prilagođeni DenseNet121, kod kog slojevi koriste prethodne osobine;
- pojednostavljeni DPN, koji kombinuje rezidualnu i gustu putanju;
- kompaktni Vision Transformer, koji sliku posmatra kao niz delova.

Cilj mi nije bio da ostvarim najbolji mogući rezultat na EuroSAT skupu, već da pod istim uslovima implementiram i uporedim četiri različita modela. Zanimalo me je koji model daje najveću test tačnost, da li veći broj parametara znači i bolji rezultat i kod kojih klasa modeli najčešće greše. Da bih to proverio, uporedio sam validacione krive, test tačnost, matrice konfuzije i rezultate po klasama. Svaki trening je izvršen jednom, pa rezultate posmatram kao poređenje ovih implementacija i hiperparametara, a ne kao konačan rang arhitektura.

1.2 Organizacija rada

U drugom poglavlju objašnjene su osnovne ideje četiri arhitekture. Zatim slede opis podataka, važni delovi implementacije i način treniranja. U šestom poglavlju prikazani su rezultati, diskusija i ograničenja, a na kraju zaključak i dva dodatka.

2 Teorijska osnova

2.1 Konvolucione neuronske mreže

Klasična potpuno povezana mreža bi sliku dimenzije $64 \times 64 \times 3$ pretvorila u dugačak vektor i izgubila deo prostorne strukture. Konvolucionni sloj čuva raspored susednih piksela i nad slikom pomera manji filter. Za jedan izlazni kanal princip se može zapisati kao

$$y_{i,j} = b + \sum_{c=1}^C \sum_{u=1}^K \sum_{v=1}^K w_{c,u,v} x_{c,i+u,j+v}, \quad (1)$$

gde su x ulaz, w težine filtera, b pomeraj, C broj ulaznih kanala, a K veličina kernela. U prvim slojevima obično se izdvajaju ivice i jednostavne teksture, dok dublji slojevi kombinuju te informacije u složenije osobine.

Posle konvolucije često se koristi batch normalizacija i ReLU aktivacija. ReLU je definisana sa

$$\text{ReLU}(x) = \max(0, x). \quad (2)$$

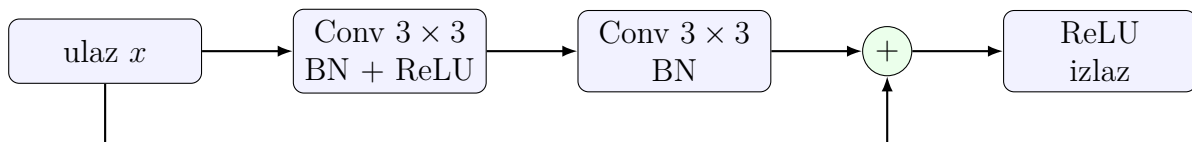
Batch normalizacija stabilizuje raspodelu aktivacija u mini-grupi i može da olakša optimizaciju dubokih mreža [1]. U svim konvolucionim modelima ovog projekta korišćene su konvolucije dimenzije 3×3 , batch normalizacija i ReLU aktivacija.

2.2 Rezidualno učenje i ResNet

Kod veoma dubokih mreža dodavanje novih slojeva ne mora automatski da poboljša rezultat. Gradijenti mogu da postanu veoma mali, a optimizacija teža. ResNet je uveo prečice, odnosno skip veze, kroz koje ulaz bloka može direktno da se doda izlazu nekoliko slojeva [2]. Osnovna relacija je

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, W) + \mathbf{x}. \quad (3)$$

Umesto da slojevi direktno uče ceo novi prikaz $\mathbf{y}$, oni uče rezidual $\mathcal{F}$. Ako se broj kanala ili prostorna dimenzija promene, prečica više ne može biti običan identitet, pa se koristi konvolucija 1×1 sa odgovarajućim korakom.



Slika 1: Pojednostavljen prikaz osnovnog rezidualnog bloka.

U ovom radu korišćena je varijanta ResNet18 prilagođena malim slikama. Početna konvolucija ne smanjuje odmah sliku velikim kernelom kao u originalnoj ImageNet verziji. Nakon toga slede četiri grupe rezidualnih blokova sa 64, 128, 256 i 512 kanala.

2.3 Guste veze i DenseNet

DenseNet uvodi drugačiju ideju: izlaz svakog sloja se ne sabira sa prethodnim izlazom, nego se konkatenira sa svim ranijim izlazima unutar istog gustog bloka [3]. Za l -ti sloj važi

$$\mathbf{x}_l = H_l([\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_{l-1}]), \quad (4)$$

gde uglaste zagrade označavaju spajanje po dimenziji kanala. Ako jedan sloj dodaje k novih kanala, broj k se naziva *growth rate*. U implementaciji je korišćeno $k = 32$.

Prednost je ponovno korišćenje već naučenih osobina. Mana je postepeni rast broja kanala, pa se između gustih blokova koriste transition slojevi. Oni smanjuju broj kanala konvolucijom 1×1 i prostornu dimenziju prosečnim udruživanjem.

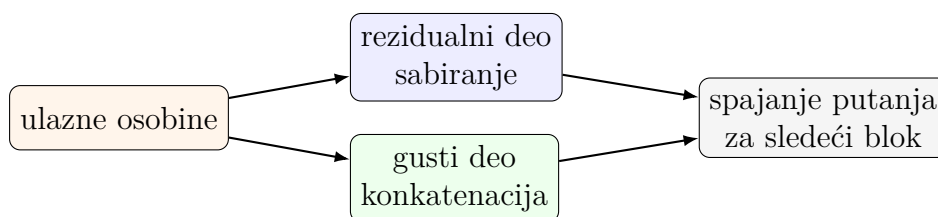
2.4 Dual Path Network

DPN kombinuje dva načina prenosa informacija [4]. Rezidualna putanja zadržava fiksni broj kanala i nove osobine dodaje sabiranjem. Gusta putanja čuva stare osobine i na njih nadovezuje nove kanale konkatenacijom. Pojednostavljeno, jedan blok može da se zapiše kao

$$\mathbf{r}_{nov} = \mathbf{r} + \Delta\mathbf{r}, \quad (5)$$

$$\mathbf{d}_{nov} = [\mathbf{d}, \Delta\mathbf{d}], \quad (6)$$

gde je $\mathbf{r}$ rezidualni, a $\mathbf{d}$ gusti deo. Važno je da se rezidualni kanali ne menjaju sa 64 na 128 prostim sabiranjem. Kada je potreban drugi broj kanala, transition sloj prvo projektuje spojene osobine u novi rezidualni prostor. Sabiraju se samo tenzori iste dimenzije.



Slika 2: Osnovna ideja dve putanje kod DPN modela.

Model napravljen u projektu predstavlja pojednostavljenu DPN varijantu prilagođenu slikama 64×64 . Zadržava suštinu dualne putanje, ali nije identična velikim DPN konfiguracijama korišćenim za ImageNet.

2.5 Vision Transformer

Vision Transformer deli sliku na male kvadratne delove, svaki deo pretvara u vektor i zatim ih obrađuje kao niz tokena [5]. Originalni rad je često prikazan primerom delova

16×16 , ali veličina dela nije obavezno uvek 16. Pošto EuroSAT slika ima samo 64×64 piksela, u ovom radu je korišćen deo 8×8 . Tako se dobija

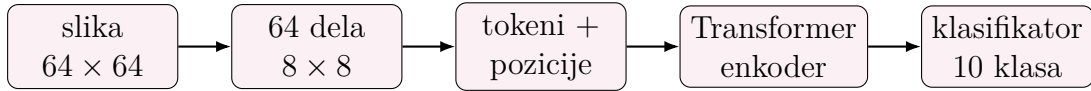
$$N = \left(\frac{64}{8}\right)^2 = 64 \quad (7)$$

delova slike. Sa delovima 16×16 dobilo bi se samo 16 tokena, što je za ovako malu sliku veoma gruba podela.

Svaki deo dobija pozicioni zapis, a na početak niza dodaje se poseban klasifikacioni token. Self-attention omogućava svakom tokenu da proceni koji su drugi delovi slike važni. Za upite Q , ključeve K i vrednosti V pažnja se računa kao

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V. \quad (8)$$

Korišćen je mali transformer: dimenzija embeddinga 128, četiri glave pažnje, dva enkoderska sloja i unutrašnja dimenzija 256. Ovakva mreža ima samo oko 0,30 miliona parametara, pa nije direktna kopija velikog ViT modela iz rada.



Slika 3: Tok podataka kroz korišćeni Vision Transformer.

2.6 Funkcija greške i mere kvaliteta

Za trening je korišćena unakrsna entropija. Ako je prava klasa y , a model za tu klasu daje verovatnoću p_y , gubitak jednog primera je

$$L = -\log(p_y). \quad (9)$$

Glavna mera za poređenje bila je tačnost

$$\text{accuracy} = \frac{\text{broj tačnih predikcija}}{\text{ukupan broj primera}}. \quad (10)$$

Za detaljniju analizu korišćene su i mere precision, recall i F1:

$$\begin{aligned} \text{precision} &= \frac{TP}{TP + FP}, \\ \text{recall} &= \frac{TP}{TP + FN}, \\ F_1 &= 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}. \end{aligned} \quad (11)$$

Precision govori koliko je predikcija određene klase bilo ispravno, recall koliko je stvarnih primera te klase pronađeno, a F1 pravi ravnotežu između te dve mere.

3 Skup podataka i pretprocesiranje

3.1 EuroSAT skup podataka

EuroSAT je javno dostupan skup namenjen klasifikaciji načina korišćenja i pokrovnosti zemljišta na osnovu satelitskih snimaka [6]. Slike su izvedene iz Sentinel-2 podataka Evropskog programa Copernicus. Puna multispektralna verzija sadrži 13 spektralnih kanala, dok je u ovom projektu korišćena RGB varijanta dostupna kroz `torchvision`. Ona sadrži 27.000 slika dimenzije 64×64 piksela.

U skupu postoji deset klasa. Broj slika po klasama nije potpuno jednak, ali nema izrazito retke klase. Najmanje primera ima klasa *Pasture*, a najveći broj klasa ima po 3.000 slika.

Tabela 1: Klase u EuroSAT RGB skupu.

Klasa	Broj slika	Klasa	Broj slika
AnnualCrop	3.000	Pasture	2.000
Forest	3.000	PermanentCrop	2.500
HerbaceousVegetation	3.000	Residential	3.000
Highway	2.500	River	2.500
Industrial	2.500	SeaLake	3.000
Ukupno			27.000

Nazivi klasa su ostavljeni na engleskom jeziku zato što se tako pojavljuju u samom skupu i u svim sačuvanim rezultatima. Na primer, *AnnualCrop* označava jednogodišnje useve, *PermanentCrop* višegodišnje zasade, dok *HerbaceousVegetation* označava zeljastu vegetaciju.

3.2 Podela na trening, validaciju i test

Split datoteke nisam sam generisao. Preuzeo sam standardne liste koje TorchGeo javno objavljuje uz EuroSAT skup na platformi Hugging Face [7]. One određuju tačno koja slika ide u trening, validacionu ili test podelu, pa su svi modeli videli iste primere. Pre treninga sam proverio da su sva imena jedinstvena i da između podela nema preklapanja. Dobijeno je 16.200, 5.400 i 5.400 slika, a odnos po svakoj klasi je približno 60/20/20, odnosno podela je približno stratifikovana. Ukupna raspodela je data u tabeli 2.

Tabela 2: Podela EuroSAT skupa korišćena u eksperimentu.

Podela	Broj slika	Udeo
Trening	16.200	60%
Validacija	5.400	20%
Test	5.400	20%
Ukupno	27.000	100%

Objekat `dataset.samples` sadrži parove putanje do slike i numeričke oznake klase. Petljom se za svako ime slike pravi rečnik `name_u_index`. Vrednost u rečniku nije nova oznaka klase, već pozicija slike u kompletnom skupu. Zatim se imena iz tekstualnih datoteka pretvaraju u te pozicije. Na primer, lista `train_indexi` može da sadrži brojeve kao 1231, 10 ili 3432; svaki broj govori koji element kompletnog skupa pripada trening podeli.

```

1 name_u_index = {}
2
3 for index, (image_path, label) in enumerate(dataset.samples):
4     image_name = Path(image_path).name
5     name_u_index[image_name] = index
6
7 def read_indexi(file_name):
8     indices = []
9     with open(split_path / file_name, "r") as file:
10         for line in file:
11             image_name = Path(line.strip()).name
12             indices.append(name_u_index[image_name])
13     return indices
14
15 train_indexi = read_indexi("euosat-train.txt")
16 val_indexi = read_indexi("euosat-val.txt")
17 test_indexi = read_indexi("euosat-test.txt")

```

Kod 1: Učitavanje indeksa slika iz unapred definisanih podela.

Trening skup služi za menjanje težina modela. Validacioni skup služi za praćenje modela tokom razvoja i izbor najbolje epohe. Test skup se koristi tek kada je trening završen. Razdvajanje ove tri uloge je važno da se test rezultat ne bi nesvesno koristio za podešavanje modela.

3.3 Pretvaranje slika i normalizacija

EuroSAT bez transformacije vraća sliku tipa `PIL.Image`. Model ne može direktno da računa sa takvim objektom, pa `ToTensor()` sliku pretvara u PyTorch tenzor oblika `[3, 64, 64]`. Tri kanala su crveni, zeleni i plavi. Pikseli, koji su prvobitno celobrojne vrednosti od 0 do 255, tada postaju realne vrednosti približno između 0 i 1.

Normalizaciju sam uradio sa srednjom vrednošću 0,5 i standardnom devijacijom 0,5 za svaki kanal:

$$x_{norm} = \frac{x - 0,5}{0,5} = 2x - 1. \quad (12)$$

Zbog toga interval `[0, 1]` prelazi približno u `[-1, 1]`. Vrednosti 0,5 nisam računao iz EuroSAT skupa, već sam ih izabrao kao jednostavan način da centriram RGB ulaz. Ista normalizacija je primenjena pri treningu, validaciji i testu.

3.4 Augmentacija trening podataka

Tokom treninga koristio sam nasumično horizontalno i vertikalno okretanje slike sa verovatnoćom 0,5. Satelitska scena nema uobičajenu orijentaciju kao, na primer, fotografija čoveka, pa okretanje uglavnom ne menja njenu klasu. Model tako kroz epohe vidi različite verzije istog primera.

Validacione i test slike se ne okreću. Njihova uloga je stabilna procena modela, pa isti primer treba da izgleda isto pri svakoj evaluaciji.

```

1 basic_transform = transforms.Compose([
2     transforms.ToTensor(),
3     transforms.Normalize(
4         (0.5, 0.5, 0.5),
5         (0.5, 0.5, 0.5)
6     )
7 ])
8
9 train_transform = transforms.Compose([
10     transforms.RandomHorizontalFlip(p=0.5),
11     transforms.RandomVerticalFlip(p=0.5),
12     basic_transform
13 ])

```

Kod 2: Transformacije za trening i evaluaciju.

Zbog različitih transformacija napravljena su dva objekta skupa. Oni pokazuju na iste slike, ali `train_dataset` primenjuje augmentaciju, dok `eval_dataset` primenjuje samo osnovnu transformaciju. Klasa `Subset` zatim bira odgovarajuće indekse za svaku podelu.

3.5 DataLoader i mini-grupe

Model ne obrađuje svih 16.200 trening slika odjednom. `DataLoader` ih grupiše u mini-grupe od po 64 slike. Jedna grupa zato ima oblik `[64, 3, 64, 64]`, a oznake oblik `[64]`. Broj 64 nije matematički obavezan; izabran je kao kompromis između potrošnje memorije i stabilnosti gradijenta.

Trening podaci se mešaju pomoću `shuffle=True`, kako redosled slika ne bi bilo moguće zapamtiti. Validacija i test ne menjaju težine, pa kod njih mešanje nije potrebno. Sa 16.200 slika i grupom 64 dobija se $\lceil 16200/64 \rceil = 254$ trening grupe po epohi.

```

1 batch_size = 64
2
3 train_loader = DataLoader(
4     train_data, batch_size=batch_size, shuffle=True
5 )
6 val_loader = DataLoader(
7     val_data, batch_size=batch_size, shuffle=False
8 )
9 test_loader = DataLoader(
10    test_data, batch_size=batch_size, shuffle=False
11 )

```

Kod 3: Formiranje tri DataLoader objekta.

4 Implementacija modela

Sva četiri modela napravljena su kao zasebne klase koje nasleđuju `torch.nn.Module`. Implementacije se nalaze u direktorijumu `src/models`. Namerno nisu uzete gotove kompletne mreže iz `torchvision.models`, jer je jedan od ciljeva projekta bio razumevanje blokova i toka tenzora. PyTorch je korišćen zbog jednostavnog definisanja slojeva, automatskog računanja gradijenata i podrške za GPU [8].

4.1 ResNet18

Osnovni blok sadrži dve konvolucije dimenzije 3×3 . Posle prve idu batch normalizacija i ReLU, a posle druge batch normalizacija. Ulaz se zatim dodaje rezultatu. Ako se menja broj kanala ili se prostorna dimenzija prepolovi, `shortcut` koristi konvoluciju 1×1 .

```
1 def forward(self, x):
2     out = self.relu(self.bn1(self.conv1(x)))
3     out = self.bn2(self.conv2(out))
4     out = out + self.shortcut(x)
5     out = self.relu(out)
6     return out
```

Kod 4: Suština forward metode rezidualnog bloka.

Mreža počinje sa 64 kanala i sadrži četiri grupe po dva bloka. Pri prelasku na novu grupu broj kanala se udvostručava, a širina i visina mape osobina se prepolove. Na kraju `AdaptiveAvgPool2d` svodi svaku mapu na jednu vrednost, a linearni sloj daje deset izlaza.

Test oblika sa slučajnim ulazom $[4, 3, 64, 64]$ dao je izlaz $[4, 10]$. Model ima 11.173.962 parametra, što ga čini najvećim modelom u ovom poređenju.

4.2 Prilagođeni DenseNet121

Jedan `DenseLayer` prvo koristi batch normalizaciju, ReLU i konvoluciju 1×1 , koja predstavlja bottleneck deo. Nakon toga slede druga batch normalizacija, ReLU i konvolucija 3×3 , koja dodaje 32 nove mape osobina. Dobijene osobine konkatenuiraju se sa ulazom.

```
1 def forward(self, x):
2     out = self.conv1(self.relu(self.bn1(x)))
3     new_features = self.conv2(self.relu(self.bn2(out)))
4     return torch.cat([x, new_features], dim=1)
```

Kod 5: Spajanje starih i novih osobina u `DenseLayer` sloju.

Na primer, ulaz $[4, 64, 64, 64]$ posle jednog sloja sa growth rate 32 postaje $[4, 96, 64, 64]$. Broj slika u grupi i prostorne dimenzije ostaju isti, a samo se broj kanala poveća za 32. U punom modelu korišćene su veličine gustih blokova 6, 12, 24 i 16, kao

kod DenseNet121, uz transition slojeve između blokova. Model zato prati DenseNet121 raspored blokova i koristi bottleneck deo, ali je prilagođen slikama 64×64 : počinje konvolucijom 3×3 , nema standardni ImageNet početni max-pooling sloj i završava se klasifikatorom za deset klasa.

Dobijeni model ima 6.956.426 parametara. Iako je dublji po broju slojeva od ResNet18, ima manje parametara jer se postojeće osobine ponovo koriste i svaki novi sloj dodaje relativno mali broj kanala.

4.3 DPN

Kod DPN bloka ulaz se sastoji od dva tenzora: rezidualnog i gustog. Oni se prvo spajaju, obrađuju zajedničkim konvolucionim slojevima, a izlaz se deli na dva dela. Prvi deo se sabira sa rezidualnom putanjom, dok se drugi dodaje gustoj putanji konkatencijom.

```

1 def forward(self, residual, dense):
2     x = torch.cat([residual, dense], dim=1)
3     features = self.layers(x)
4
5     residual_update = features[:, :self.residual_channels]
6     dense_update = features[:, self.residual_channels:]
7
8     new_residual = residual + residual_update
9     new_dense = torch.cat([dense, dense_update], dim=1)
10    return new_residual, new_dense

```

Kod 6: Pojednostavljena dualna putanja u DPN bloku.

Na početku postoje 64 rezidualna i 16 gustih kanala. Posle prvog bloka gusta putanja ima 32, a posle drugog 48 kanala, dok rezidualna ostaje na 64. Transition sloj zato spojenih 112 kanala pretvara u 128 rezidualnih i 16 gustih kanala, uz smanjenje prostorne dimenzije sa 64×64 na 32×32 . To objašnjava zašto promena sa 64 na 128 nije obično sabiranje kanala.

Puni DPN u ovom projektu ima 9.778.522 parametra. Konfiguracija je manja od originalnih DPN mreža, ali je dovoljna da se praktično ispita osnovna ideja rezidualne i guste putanje.

4.4 Kompaktni Vision Transformer

Sloj za pravljenje delova slike realizovan je konvolucijom čiji su kernel i korak jednaki veličini dela. Za deo 8, konvolucija u jednom prolazu pretvara 8×8 RGB blok u vektor embeddinga dimenzije 128.

```

1 def forward(self, x):
2     x = self.projection(x)           # [B, 128, 8, 8]
3     x = x.flatten(2)                 # [B, 128, 64]
4     x = x.transpose(1, 2)            # [B, 64, 128]
5     return x

```

Kod 7: Pretvaranje slike u niz tokena kod ViT modela.

Nakon toga se dodaju klasifikacioni token i pozicioni embedding. Niz prolazi kroz dva transformer encoder sloja. Za klasifikaciju se uzima samo izlaz klasifikacionog tokena, normalizuje se i prosleđuje linearnom sloju sa deset izlaza.

Izlaz za ulaz $[2, 3, 64, 64]$ bio je $[2, 10]$, a model ima 299.658 parametara. To je približno 23 puta manje parametara od DenseNet121, pa je ViT u ovom radu namerno veoma kompaktan.

4.5 Poređenje veličine modela

Tabela 3: Broj parametara implementiranih modela.

Model	Broj parametara	Približno miliona
ResNet18	11.173.962	11,17
DenseNet121	6.956.426	6,96
DPN	9.778.522	9,78
ViT	299.658	0,30

Broj parametara govori o veličini modela, ali ne određuje automatski tačnost. ResNet18 je najveći, ali nije pobedio. Prilagođeni DenseNet121 je ostvario najbolji rezultat sa približno 38% manje parametara od ResNet18. ViT je najmanji, ali je posle deset epoha imao i najnižu tačnost.

5 Postavka eksperimenta i treniranje

5.1 Softversko i hardversko okruženje

Razvoj i početne provere uradio sam lokalno u Visual Studio Code-u. Prvi pokušaj treniranja na procesoru stigao je samo do grupe 25 od 254 za oko sedam minuta, pa bi treniranje sva četiri modela trajalo predugo. Završni eksperimenti prvobitno su zato izvršeni u Google Colab okruženju zbog dostupnosti NVIDIA Tesla T4 grafičke kartice. Nakon treniranja notebook je sreden i prilagođen strukturi lokalnog projekta, dok su dobijeni rezultati sačuvani.

Tokom rada mi je jednom istekao Colab runtime pre nego što je ResNet checkpoint prebačen na lokalni računar. Taj model sam ponovo istrenirao, a posle toga sam checkpoint i rezultate preuzimao odmah po završetku svakog modela. To je i razlog što su u repozitorijumu sačuvane metrike i slike, dok su veliki checkpoint fajlovi ostali lokalno.

Sačuvani izlaz iz Colab treninga beleži PyTorch 2.11.0+cu128, dostupnu CUDA podršku i Tesla T4 grafičku karticu. Konačni notebook je zatim sreden u lokalnom Python 3.13.3 okruženju. U njemu su korišćene i biblioteke torchvision, NumPy, Matplotlib, pandas i scikit-learn. Spisak potrebnih paketa za ponovno pokretanje nalazi se u datoteci `requirements.txt`. Izvorni kod, podele, notebook fajlovi i rezultati su na adresi https://github.com/markovicdjordje-rs/DPN_EuroSAT [9].

Sačuvani rezultati u notebooku potiču iz originalnog Colab izvršavanja, pa se u pojedinim starijim ispisima vidi putanja `/content/DPN_EuroSAT/splits`. U konačnom

kodu putanje su prilagođene lokalnoj strukturi projekta i podele se čitaju iz direktorijuma `data/splits`.

5.2 Zajednički hiperparametri

Da bi poređenje bilo pregledno, za sve modele korišćena je ista osnovna postavka. Hiperparametri su dati u tabeli 4.

Tabela 4: Hiperparametri korišćeni za sva četiri modela.

Hiperparametar	Vrednost
Broj epoha	10
Veličina mini-grupe	64
Optimizator	Adam
Početna stopa učenja	0,001
Funkcija greške	CrossEntropyLoss
Broj klasa	10
Seed	42
Augmentacija	horizontalno i vertikalno okretanje

Adam kombinuje adaptivne stope učenja sa momentom i često daje dobar početni rezultat bez dugog ručnog podešavanja [10]. U ovom projektu nije korišćen scheduler za promenu stope učenja. To pojednostavljuje kod, ali je verovatno jedan od razloga što validaciona tačnost nekih modela osciluje.

5.3 Jedna epoha treninga

Na početku epohe poziva se `model.train()`. Batch normalizacija, a kod ViT modela i dropout, tada prelaze u režim treninga. Za svaku mini-grupu obavlja se isti niz koraka:

1. slike i oznake se prebacuju na GPU;
2. brišu se stari gradijenti;
3. model računa izlaze;
4. računa se greška;
5. `backward()` računa gradijente;
6. optimizator menja parametre;
7. broje se tačne predikcije.

```

1 model.train()
2 train_correct = 0
3
4 for images, labels in train_loader:
5     images = images.to(device)
6     labels = labels.to(device)
7
8     optimizer.zero_grad()
9     outputs = model(images)
10    loss = loss_function(outputs, labels)
11
12    loss.backward()
13    optimizer.step()
14
15    predictions = outputs.argmax(dim=1)
16    train_correct += (predictions == labels).sum().item()
17
18 train_accuracy = 100 * train_correct / len(train_data)

```

Kod 8: Glavni deo petlje za treniranje jedne epohe.

Linija `outputs.argmax(dim=1)` za svaku sliku bira indeks najvećeg od deset izlaza. Poređenjem tog indeksa sa pravom oznakom dobija se broj tačnih predikcija. Trening tačnost se zatim deli ukupnim brojem trening slika, a ne brojem mini-grupa.

5.4 Validacija i čuvanje najboljeg modela

Posle trening dela epohe model se prebacuje u `eval()` režim. Gradijenti se ne računaju, jer se na validacionim podacima težine ne menjaju. To štedi memoriju i ubrzava evaluaciju.

```

1 model.eval()
2 val_correct = 0
3
4 with torch.no_grad():
5     for images, labels in val_loader:
6         images = images.to(device)
7         labels = labels.to(device)
8         outputs = model(images)
9         predictions = outputs.argmax(dim=1)
10        val_correct += (predictions == labels).sum().item()
11
12 val_accuracy = 100 * val_correct / len(val_data)
13
14 if val_accuracy > best_val_accuracy:
15     best_val_accuracy = val_accuracy
16     torch.save(model.state_dict(), checkpoint_path)

```

Kod 9: Validacija i čuvanje najboljeg checkpointa.

Ne čuva se samo model iz poslednje epohe, nego model iz epohe sa najvećom validacionom tačnošću. Ovo je važno jer se, na primer, DenseNet validaciona tačnost smanjila sa 91,44% u osmoj epohi na 70,39% u desetoj. Da je korišćen samo poslednji

model, završni rezultat bi bio značajno lošiji.

Za testiranje se pravi nov objekat odgovarajućeg modela, učitava se sačuvani `state_dict`, model prelazi u `eval()` režim i tek tada se računa test tačnost. Test skup ni u jednom trenutku ne utiče na izbor checkpointa.

5.5 Ponovljivost eksperimenta

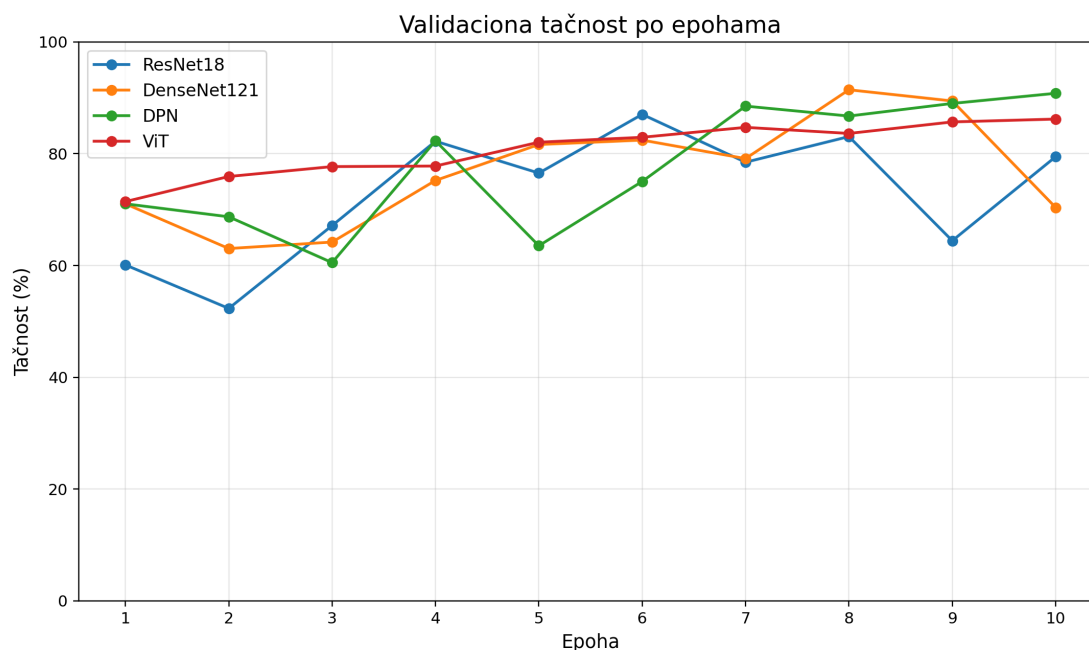
Postavljen je `torch.manual_seed(42)`. To pomaže da početne težine i deo nasumičnih operacija budu ponovljivi, ali ne garantuje potpuno identičan rezultat na svakoj kombinaciji GPU-a, CUDA biblioteke i verzije PyTorch-a. Pored toga, nasumične augmentacije menjaju ulaze kroz epohe. Zato male razlike između dva pokretanja istog treninga nisu neočekivane.

U repozitorijumu su sačuvani JSON fajlovi sa trening i validacionim tačnostima, CSV tabela poređenja, klasifikacioni izveštaji i slike grafikona. Time je rezultat moguće analizirati bez ponovnog treninga svih modela.

6 Rezultati i analiza

6.1 Kretanje validacione tačnosti

Slika 4 prikazuje validacionu tačnost kroz deset epoha. Krive ne rastu potpuno ravnomerno. To je posebno vidljivo kod ResNet, DenseNet i DPN modela. Razlog može biti relativno velika fiksna stopa učenja, augmentacija i činjenica da je svaki model treniran samo deset epoha.



Slika 4: Validaciona tačnost sva četiri modela po epohama.

ResNet18 dostiže najbolju validacionu tačnost od 87,04% u šestoj epohi, ali posle

toga osciluje. DenseNet121 ima najbolji pojedinačni validacioni rezultat, 91,44%, u osmoj epohi. DPN se popravljajući sporije i najbolji rezultat od 90,81% dostiže u desetoj epohi. ViT ima mirniju krivu i postepeno raste do 86,20%, ali ostaje ispod najboljih konvolucionih modela.

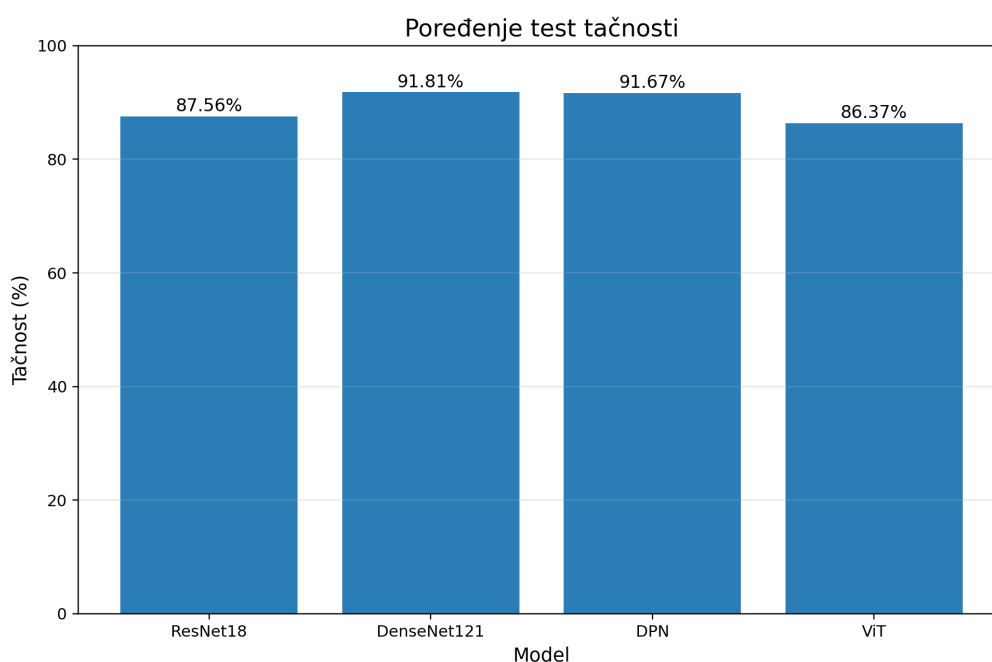
Validaciona tačnost ponekad je veća od trening tačnosti. Trening slike su nasumično okretane, pa je taj zadatak malo teži. Batch normalizacija, a kod ViT modela i dropout, takođe se drugačije ponašaju u režimima `train()` i `eval()`.

6.2 Konačna test tačnost

Za svaki model na test skupu je korišćen checkpoint sa najboljom validacionom tačnošću. Rezultati su dati u tabeli 5 i na slici 5.

Tabela 5: Glavni rezultati eksperimenta.

Model	Parametri	Najbolja epoha	Najbolja val.	Test
ResNet18	11.173.962	6	87,04%	87,56%
DenseNet121	6.956.426	8	91,44%	91,81%
DPN	9.778.522	10	90,81%	91,67%
ViT	299.658	10	86,20%	86,37%



Slika 5: Poređenje test tačnosti modela.

DenseNet121 je najbolji sa 91,81%, ali je DPN veoma blizu sa 91,67%. Razlika je samo 0,14 procentnih poena, odnosno oko osam slika na test skupu od 5.400 primera. Zbog toga nije opravdano tvrditi da je DenseNet uopšteno mnogo bolji; u ovom konkretnom pokretanju je samo imao malo veći rezultat.

ResNet18 zaostaje približno 4,25 procentnih poena za DenseNetom, iako ima više parametara. Rezultat pokazuje da veći broj parametara sam po sebi nije doneo bolju tačnost. Moguće je da su guste veze i ponovno korišćenje osobina doprineli boljem rezultatu prilagođenog DenseNeta, ali to ovim eksperimentom nije posebno izolovano.

ViT ima najnižu test tačnost, ali i daleko najmanje parametara. Originalni ViT rezultati se u velikoj meri oslanjaju na prethodno treniranje na veoma velikim skupovima [5]. Moj mali transformer je treniran od nule na 16.200 slika samo deset epoha i ostvario je 86,37%.

6.3 Matrice konfuzije

Matrica konfuzije pokazuje raspodelu predikcija po klasama. Red predstavlja stvarnu klasu, a kolona predviđenu klasu. Vrednosti su normalizovane po redovima, pa je idealna matrica ona kod koje su sva polja na glavnoj dijagonali blizu 1.

Kod sva četiri modela veoma dobro se prepoznaju klase *Residential*, *Forest* i *SeaLake*. One imaju relativno prepoznatljive teksture: gusto raspoređene zgrade, šumski pokrivač ili velike vodene površine.

Teže su klase koje dele slične boje i šare. *HerbaceousVegetation* se može pomešati sa *PermanentCrop*, dok se *Highway* i *River* ponekad mešaju sa klasama koje sadrže izdužene strukture. ResNet posebno slabije prepoznaje *Highway* i *River*. DenseNet i DPN imaju tamniju dijagonalu za većinu klasa, što je u skladu sa njihovom većom ukupnom tačnošću.

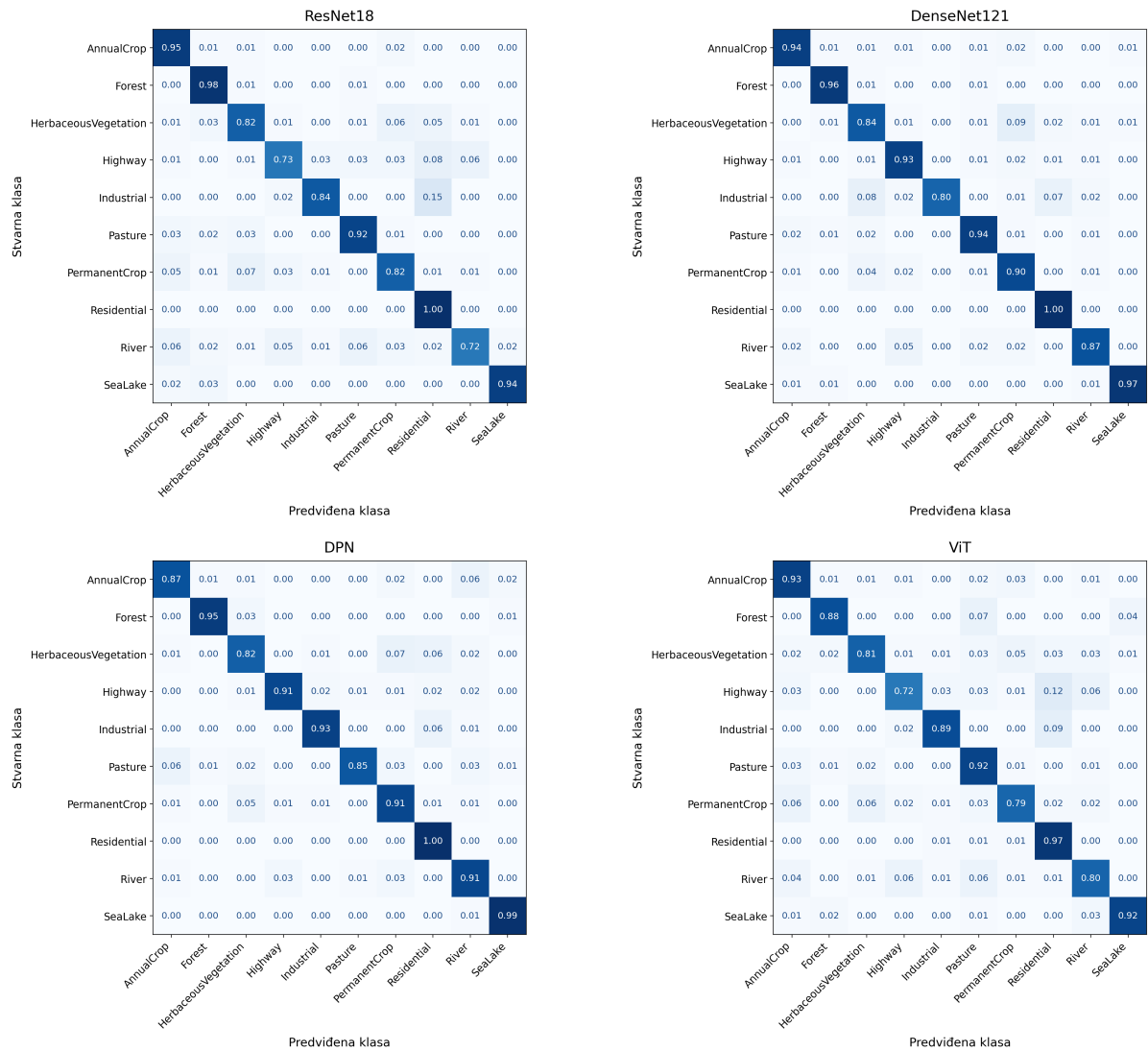
Kod DPN-a je recall za *Residential* skoro 1,00. ViT takođe vrlo dobro prepoznaje tu klasu, ali kod *Highway* ima recall oko 0,72. To pokazuje da ukupna tačnost sama ne otkriva gde tačno model greši.

6.4 Detaljniji rezultati DPN i ViT modela

DPN je centralni model projekta, a ViT predstavlja dodatno poređenje sa drugačijom vrstom arhitekture. Zato su za njih sačuvani potpuni klasifikacioni izveštaji. Tabela 6 sadrži rezultate DPN-a.

Tabela 6: Rezultati DPN modela po klasama na test skupu.

Klasa	Precision	Recall	F1	Broj
AnnualCrop	0,920	0,874	0,897	596
Forest	0,990	0,949	0,969	608
HerbaceousVegetation	0,872	0,822	0,846	573
Highway	0,949	0,907	0,928	496
Industrial	0,949	0,932	0,941	501
Pasture	0,971	0,854	0,909	396
PermanentCrop	0,849	0,911	0,879	538
Residential	0,878	0,998	0,934	554
River	0,856	0,913	0,884	529
SeaLake	0,958	0,985	0,972	609
Makro prosek	0,919	0,915	0,916	5.400
Težinski prosek	0,919	0,917	0,917	5.400



Slika 6: Normalizovane matrice konfuzije za sva četiri modela.

Najveći DPN F1 rezultat imaju *SeaLake* i *Forest*. Kod klase *Residential* recall je 0,998, ali je precision 0,878. To znači da je model pronašao skoro sve stvarne stambene zone, ali je i deo drugih slika pogrešno proglasio za stambene. Klasa *HerbaceousVegetation* ima najniži F1, 0,846, što se vidi i u matrici konfuzije.

Tabela 7: Rezultati ViT modela po klasama na test skupu.

Klasa	Precision	Recall	F1	Broj
AnnualCrop	0,852	0,926	0,887	596
Forest	0,943	0,878	0,910	608
HerbaceousVegetation	0,905	0,813	0,857	573
Highway	0,849	0,716	0,777	496
Industrial	0,929	0,886	0,907	501
Pasture	0,710	0,917	0,800	396
PermanentCrop	0,871	0,790	0,828	538
Residential	0,801	0,975	0,879	554
River	0,832	0,798	0,815	529
SeaLake	0,946	0,924	0,935	609
Makro prosek	0,864	0,862	0,860	5.400
Težinski prosek	0,870	0,864	0,864	5.400

Kod ViT-a se primećuje visok recall, a niži precision za *Pasture* i *Residential*. Model te oznake dodeljuje previše često. Najslabiji F1 je za *Highway*, 0,777. Sa druge strane, *SeaLake*, *Forest* i *Industrial* ostaju prilično pouzdane klase.

6.5 Primeri DPN predikcija

Na slici 7 prikazano je šest primera iz test skupa. Naslov svakog primera sadrži stvarnu i predviđenu klasu. U svih šest prikazanih slučajeva DPN je dao tačan odgovor.



Slika 7: Primeri tačnih DPN predikcija na test slikama.

Na originalnoj dimenziji od 64×64 detalji su mali. Ipak, tekstura šume, jednolična vodena površina, pravolinijski usevi i gusta izgradnja daju dovoljno informacija za klasifikaciju.

6.6 Odnos tačnosti i broja parametara

Tabela 8: Grubo poređenje tačnosti i veličine modela.

Model	Mil. parametara	Test tačnost	Kratak komentar
ResNet18	11,17	87,56%	najveći model, ali treći rezultat
DenseNet121	6,96	91,81%	najbolja test tačnost
DPN	9,78	91,67%	skoro isti rezultat kao DenseNet
ViT	0,30	86,37%	najmanji model, treniran od nule

Prilagođeni DenseNet121 se pokazao kao najbolji kompromis. DPN je vrlo blizu po tačnosti, ali ima približno 2,82 miliona parametara više. ViT ima ubedljivo najmanje parametara. To ga čini zanimljivim za uređaje sa ograničenom memorijom, ali nisam merio brzinu izvršavanja, pa o njoj ne mogu da izvedem siguran zaključak.

6.7 Diskusija glavnih nalaza

Najbolji rezultat ima prilagođeni DenseNet121 sa 91,81%, ali DPN praktično deli prvo mesto sa 91,67%. Jedno moguće objašnjenje jeste da guste veze pomažu ponovnom korišćenju osobina, ali bi za sigurniji zaključak bilo potrebno više ponavljanja.

Veći broj parametara nije garantovao bolji rezultat. ResNet18 ima najviše parametara, ali ga DenseNet i DPN nadmašuju, dok najmanji ViT takođe nije prvi. Na rezultat su zajedno uticali arhitektura, podaci i način treninga.

Po klasama, najlakše su uglavnom *SeaLake*, *Forest* i *Residential*. Teže su klase sa sličnim zelenim ili poljoprivrednim teksturama, kao i *Highway* kod ViT-a. Matrice konfuzije i F1 mere su zato korisnije od same ukupne tačnosti.

Kod sva četiri modela test tačnost je malo veća od najbolje validacione. Validacioni i test skup sadrže različite slike, pa test skup u ovom slučaju može biti nešto lakši za klasifikaciju. Zbog toga mala razlika u korist test tačnosti nije neočekivana.

6.8 Ograničenja

Eksperiment ima nekoliko ograničenja:

- Svaki model je treniran samo jednom i samo deset epoha. Bez više ponavljanja nije moguće proceniti standardnu devijaciju rezultata.
- Za sve modele je korišćena ista stopa učenja. To čini poređenje jednostavnim, ali možda nije optimalno za svaku arhitekturu.
- Nije korišćen scheduler, rano zaustavljanje niti detaljna pretraga hiperparametara. Validacione krive zato imaju vidljive oscilacije.
- Modeli su trenirani od nule. Posebno ViT obično dobija veliku korist od prethodnog treniranja na većem skupu.
- Korišćena su samo tri RGB kanala, iako Sentinel-2 sadrži dodatne spektralne kanale korisne za razlikovanje vegetacije i zemljišta.
- Nisu merena vremena treninga, inferencije niti tačna potrošnja GPU memorije. Broj parametara daje samo delimičnu sliku efikasnosti.

Zbog ovih ograničenja rezultat ne treba posmatrati kao dokaz da će prilagođeni DenseNet uvek biti najbolji, već kao rezultat za ovu podelu i ove hiperparametre.

7 Zaključak

U radu su implementirana i poređena četiri modela za klasifikaciju EuroSAT satelitskih snimaka. Ceo tok je obuhvatio učitavanje skupa, povezivanje imena slika sa unapred definisanim podelama, transformacije, formiranje mini-grupa, implementaciju mreža, trening na GPU-u, čuvanje najboljih checkpointa i završnu analizu.

Najbolju test tačnost ostvario je prilagođeni DenseNet121 sa 91,81%. DPN je sa 91,67% pokazao da kombinovanje rezidualne i guste putanje dobro radi na ovom zadatku. ResNet18 je ostvario 87,56%, dok je mali ViT ostvario 86,37%. Iako je ViT poslednji po tačnosti, imao je samo 299.658 parametara, pa je njegov rezultat zanimljiv u odnosu na veličinu modela.

Praktično sam video da veći model nije automatski bolji. DenseNet je sa manje parametara nadmašio ResNet, dok je DPN bio skoro jednako dobar kao DenseNet. Izbor

najboljeg checkpointa po validacionoj tačnosti bio je neophodan zbog oscilacija kroz epohe.

Kao nastavak rada bilo bi korisno trenirati modele više epoha uz scheduler, ponoviti svaki eksperiment sa više seed vrednosti i prikazati srednju vrednost i standardnu devijaciju. Za ViT bi trebalo proveriti prethodno trenirane težine i veće konfiguracije, a za sve modele ispitati multispektralne Sentinel-2 kanale. Moguće je probati i jaču augmentaciju, kao i ansambl DenseNet i DPN modela.

A Struktura projekta

Projekat je organizovan tako da su kod, notebook fajlovi, podele i rezultati odvojeni. Pojednostavljena struktura izgleda ovako:

```
DPN_EuroSAT/
|-- data/
|   '-- splits/
|-- notebooks/
|   |-- 01_eurosat_analysis.ipynb
|   '-- 02_train_final.ipynb
|-- results/
|   |-- figures/
|   '-- metrics/
|-- src/
|   '-- models/
|       |-- resnet.py
|       |-- densenet.py
|       |-- dpn.py
|       '-- vit.py
|-- README.md
'-- requirements.txt
```

Kod 10: Osnovna struktura repozitorijuma.

Direktorijum `data/raw` i checkpoint fajlovi nisu postavljeni na GitHub zbog veličine. Dataset se može ponovo preuzeti kroz `torchvision`, a rezultati u `results/metrics` i `results/figures` omogućavaju pregled bez ponovnog treninga. Konačna verzija projekta dostupna je na https://github.com/markovicdjordje-rs/DPN_EuroSAT.

Za lokalno pokretanje potrebno je napraviti virtuelno okruženje, instalirati pakete iz `requirements.txt` i otvoriti notebook iz direktorijuma `notebooks`. Koren projekta se dodaje u `sys.path` kako bi uvoz iz `src.models` radio ispravno:

```
1 from pathlib import Path
2 import sys
3
4 project_path = Path.cwd()
5 if project_path.name == "notebooks":
6     project_path = project_path.parent
7
8 sys.path.insert(0, str(project_path))
9
10 data_path = project_path / "data" / "raw"
11 split_path = project_path / "data" / "splits"
```

Kod 11: Određivanje putanja u lokalnom notebook okruženju.

B Tačnosti po epohama

Tabela 9 čuva vrednosti sa validacionog grafikona. Ove vrednosti su sačuvane u JSON fajlovima za svaki model.

Tabela 9: Validaciona tačnost po epohama u procentima.

Epoha	ResNet18	DenseNet121	DPN	ViT
1	60,11	71,04	71,04	71,44
2	52,33	63,04	68,72	75,93
3	67,15	64,20	60,52	77,70
4	82,24	75,22	82,31	77,80
5	76,54	81,65	63,54	82,06
6	87,04	82,44	75,04	82,94
7	78,48	79,13	88,52	84,72
8	83,07	91,44	86,74	83,63
9	64,44	89,44	89,00	85,69
10	79,46	70,39	90,81	86,20

Tabela jasno pokazuje zašto se poslednja epoha ne sme automatski uzeti kao najbolja. Kod DenseNet-a razlika između najbolje i poslednje validacione tačnosti iznosi više od 21 procentni poen.

Literatura

- [1] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proceedings of the 32nd International Conference on Machine Learning*, pp. 448–456, 2015.
- [2] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770–778, 2016.
- [3] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, “Densely connected convolutional networks,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 4700–4708, 2017.
- [4] Y. Chen, J. Li, H. Xiao, X. Jin, S. Yan, and J. Feng, “Dual path networks,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [5] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An image is worth 16x16 words: Transformers for image recognition at scale,” in *International Conference on Learning Representations*, 2021.
- [6] P. Helber, B. Bischke, A. Dengel, and D. Borth, “EuroSAT: A novel dataset and deep learning benchmark for land use and land cover classification,” *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 12, no. 7, pp. 2217–2226, 2019.
- [7] TorchGeo contributors, “EuroSAT train, validation and test split files.” Hugging Face repozitorijum. Dostupno na: <https://huggingface.co/datasets/torchgeo/eurosat/tree/main>, 2026. Pristupljeno: septembar 2026.
- [8] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, vol. 32, pp. 8024–8035, 2019.
- [9] Đ. Marković, “DPN_EuroSAT: poređenje modela za klasifikaciju satelitskih snimaka.” GitHub repozitorijum. Dostupno na: https://github.com/markovicdjordje-rs/DPN_EuroSAT, 2026. Pristupljeno: septembar 2026.
- [10] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *International Conference on Learning Representations*, 2015.